

고객을 세그멘테이션하자 [프로젝트] - 송경섭

11-2. 데이터 불러오기

데이터 살펴보기

```
mysql( """
SELECT *
FROM data_df
LIMIT 10;
""" )
```

- 테이블에 있는 10개의 행만 출력하기

[결과 이미지를 넣어주세요]

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850.0	United Kingdom
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850.0	United Kingdom
7	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
8	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	1.69	13047.0	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
# 2) 전체 데이터는 몇 행으로 구성되어 있는지 확인하기
mysql( """
SELECT COUNT(*) AS row_cnt
FROM data_df
""" )
[결과 이미지를 넣어주세요]
```

row_cnt
0 541909

데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
mysql( """
SELECT
    COUNT(InvoiceNo) AS cnt_InvoiceNo,
    COUNT(StockCode) AS cnt_StockCode,
    COUNT(Description) AS cnt_Description,
    COUNT(Quantity) AS cnt_Quantity,
    COUNT(InvoiceDate) AS cnt_InvoiceDate,
    COUNT(UnitPrice) AS cnt_UnitPrice,
    COUNT(CustomerID) AS cnt_CustomerID,
    COUNT(Country) AS cnt_Country
FROM data_df
""" )
```

[결과 이미지를 넣어주세요]

cnt InvoiceNo	cnt StockCode	cnt Description	cnt Quantity	cnt InvoiceDate	cnt UnitPrice	cnt CustomerID	cnt Country
541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율을 계산
 - 각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기

```
mysql("""
SELECT
    'InvoiceNo' AS column_name,
    SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'StockCode' AS column_name,
    SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'Description' AS column_name,
    SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'Quantity' AS column_name,
    SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'InvoiceDate' AS column_name,
    SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
```

```

SELECT
    'UnitPrice' AS column_name,
    SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'CustomerID' AS column_name,
    SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df

UNION ALL
SELECT
    'Country' AS column_name,
    SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) AS missing_cnt,
    COUNT(*) AS total_cnt,
    ROUND(1.0 * SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*), 6) AS missing_ratio,
    ROUND(100.0 * SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) || '%' AS missing_pct
FROM data_df;
""")

```

[결과 이미지를 넣어주세요]

	column_name	missing_cnt	total_cnt	missing_ratio	missing_pct
0	InvoiceNo	0	541909	0.000000	0.0%
1	StockCode	0	541909	0.000000	0.0%
2	Description	1454	541909	0.002683	0.27%
3	Quantity	0	541909	0.000000	0.0%
4	InvoiceDate	0	541909	0.000000	0.0%
5	UnitPrice	0	541909	0.000000	0.0%
6	CustomerID	135080	541909	0.249267	24.93%
7	Country	0	541909	0.000000	0.0%

결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```

mysql("""
SELECT Description
FROM data_df
WHERE StockCode = '85123A'
""")

```

[결과 이미지를 넣어주세요]

	Description
0	WHITE HANGING HEART T-LIGHT HOLDER
1	WHITE HANGING HEART T-LIGHT HOLDER
2	WHITE HANGING HEART T-LIGHT HOLDER
3	WHITE HANGING HEART T-LIGHT HOLDER
4	WHITE HANGING HEART T-LIGHT HOLDER
...	...
2308	CREAM HANGING HEART T-LIGHT HOLDER
2309	WHITE HANGING HEART T-LIGHT HOLDER
2310	WHITE HANGING HEART T-LIGHT HOLDER
2311	CREAM HANGING HEART T-LIGHT HOLDER
2312	CREAM HANGING HEART T-LIGHT HOLDER

결측치 처리

- DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시

```
mysql("""
SELECT COUNT(*) AS missing_customerid
FROM clean_df
WHERE CustomerID IS NULL
""")
```

[결과 이미지를 넣어주세요]

missing_customerid
0
0

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- 중복된 행의 수를 세어보기
 - 8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기

```
mysql("""
SELECT
    SUM(cnt - 1) AS duplicate_row_cnt
FROM (
    SELECT
        InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country,
        COUNT(*) AS cnt
    FROM clean_df
    GROUP BY
        InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
    HAVING COUNT(*) > 1
)
```

```
) t  
"""
```

[결과 이미지를 넣어주세요]

duplicate_row_cnt	
0	5225

중복값 처리

- 중복값을 제거하는 쿼리문 작성하기
 - CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

```
distinct_df = mysql("""  
SELECT DISTINCT *  
FROM clean_df  
""")  
print("before:", clean_df.shape, "after:", distinct_df.shape)
```

[결과 이미지를 넣어주세요]

```
before: (406829, 8) after: (401604, 8)
```

11-6. 데이터 전처리(3): 오류값 처리

InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo 의 개수를 출력하기

```
mysql("""  
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoiceno_cnt  
FROM distinct_df  
""")
```

[결과 이미지를 넣어주세요]

unique_invoiceno_cnt	
0	22190

- 고유한 InvoiceNo 를 앞에서부터 100개를 출력하기

```
mysql("""  
SELECT DISTINCT InvoiceNo  
FROM distinct_df  
ORDER BY InvoiceNo  
LIMIT 100  
""")
```

[결과 이미지를 넣어주세요]

InvoiceNo	
0	536365
1	536366
2	536367
3	536368
4	536369
...	...
95	536569
96	536570
97	536571
98	536572
99	536573
100 rows × 1 columns	

- **InvoiceNo** 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
mysql("""
SELECT *
FROM data_df
WHERE InvoiceNo LIKE 'C%'
LIMIT 100
""")
```

[결과 이미지를 넣어주세요]

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	C536379	D	Discount	-1	12/1/2010 9:41	27.50	14527.0	United Kingdom
1	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	12/1/2010 9:49	4.65	15311.0	United Kingdom
2	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	12/1/2010 10:24	1.65	17548.0	United Kingdom
3	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	12/1/2010 10:24	0.29	17548.0	United Kingdom
4	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	12/1/2010 10:24	0.29	17548.0	United Kingdom
...	...	...	...	...	...	...	...	...
95	C537143	20685	DOORMAT RED RETROSPOT	-1	12/5/2010 12:58	7.95	12748.0	United Kingdom
96	C537157	35953	FOLKART STAR CHRISTMAS DECORATIONS	-24	12/5/2010 13:09	1.25	15880.0	United Kingdom
97	C537164	D	Discount	-1	12/5/2010 13:21	29.29	14527.0	United Kingdom
98	C537203	37449	CERAMIC CAKE STAND + HANGING CAKES	-1	12/5/2010 14:44	9.95	14606.0	United Kingdom
99	C537203	21424	WOODLAND STORAGE BOX LARGE	-1	12/5/2010 14:44	2.95	14606.0	United Kingdom

- 구매 건 상태가 **Canceled** 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
mysql("""
SELECT
ROUND(
ROUND(
100.0 * SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*),
1
) AS canceled_ratio_pct
FROM distinct_df
""")
```

[결과 이미지를 넣어주세요]

canceled_ratio_pct
0 2.2

StockCode 살펴보기

- 고유한 **StockCode** 의 개수를 출력하기

```
mysql("""
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode_cnt
FROM distinct_df
""")
```

[결과 이미지를 넣어주세요]

	unique_stockcode_cnt
0	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 **StockCode** 별 등장 빈도를 출력하기

- 상위 10개의 제품들을 출력하기

```
mysql("""
SELECT
    StockCode,
    COUNT(*) AS sell_cnt
FROM distinct_df
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10
""")
```

[결과 이미지를 넣어주세요]

	StockCode	sell_cnt
0	85123A	2065
1	22423	1894
2	85099B	1659
3	47566	1409
4	84879	1405
5	20725	1346
6	22720	1224
7	POST	1196
8	22197	1110
9	23203	1108

- StockCode** 의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고

- 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지 출력하기

```
mysql("""
SELECT DISTINCT
    StockCode,
    (LENGTH(StockCode) - LENGTH(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
    StockCode,
    '0',''), '1',''), '2',''), '3',''), '4',''), '5',''), '6',''), '7',''), '8',''), '9',''))
```

```

    )) AS number_count
FROM distinct_df
WHERE (LENGTH(StockCode) - LENGTH(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
    REPLACE(REPLACE(
        StockCode,
        '0',''), '1',''), '2',''), '3',''), '4',''), '5',''), '6',''), '7',''), '8',''), '9',''))
    )) IN (0, 1)
ORDER BY number_count, StockCode
""")

```

[결과 이미지를 넣어주세요]

	StockCode	number_count
0	BANK CHARGES	0
1	CRUK	0
2	D	0
3	DOT	0
4	M	0
5	PADS	0
6	POST	0
7	C2	1

- **StockCode**의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고
 - 숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트인지 구하기 (소수점 두 번째 자리까지)

```

mysql("""
WITH t AS (
    SELECT
        StockCode,
        LENGTH(StockCode) - LENGTH(
            REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
                StockCode,
                '0',''), '1',''), '2',''), '3',''), '4',''), '5',''), '6',''), '7',''), '8',''), '9',''))
        ) AS number_count
    FROM distinct_df
)
SELECT
    ROUND(
        100.0 * SUM(CASE WHEN number_count IN (0, 1) THEN 1 ELSE 0 END) / COUNT(*),
        2
    ) AS pct_numbercount_0_1
FROM t
""")

```

[결과 이미지를 넣어주세요]

pct_numbercount_0_1
0 0.48

- 제품과 관련되지 않은 거래 기록을 제거하기


```

filtered_df = mysql("""
SELECT *
FROM distinct_df
WHERE StockCode NOT IN (
    SELECT DISTINCT StockCode
    FROM (
        SELECT
            StockCode,
            LENGTH(StockCode) - LENGTH(
                REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
                    StockCode,
                    '0',''), '1',''), '2',''), '3',''), '4',''), '5',''), '6',''), '7',''), '8',''), '9','')
            ) AS number_count
        FROM distinct_df
    ) t
    WHERE number_count IN (0, 1)
)
""")

print("before:", distinct_df.shape, "after:", filtered_df.shape)

```

[결과 이미지를 넣어주세요]

before: (401604, 8) after: (399689, 8)

Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```

mysql("""
SELECT
    Description,
    COUNT(*) AS description_cnt
FROM filtered_df
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30
""")

```

[결과 이미지를 넣어주세요]

	Description	description_cnt
0	WHITE HANGING HEART T-LIGHT HOLDER	2058
1	REGENCY CAKESTAND 3 TIER	1894
2	JUMBO BAG RED RETROSPOT	1659
3	PARTY BUNTING	1409
4	ASSORTED COLOUR BIRD ORNAMENT	1405
5	LUNCH BAG RED RETROSPOT	1345
6	SET OF 3 CAKE TINS PANTRY DESIGN	1224
7	LUNCH BAG BLACK SKULL	1099
8	PACK OF 72 RETROSPOT CAKE CASES	1062
9	SPOTTY BUNTING	1026
10	PAPER CHAIN KIT 50'S CHRISTMAS	1013
11	LUNCH BAG SPACEBOY DESIGN	1006
12	LUNCH BAG CARS BLUE	1000
13	HEART OF WICKER SMALL	990
14	NATURAL SLATE HEART CHALKBOARD	989
15	JAM MAKING SET WITH JARS	966
16	LUNCH BAG PINK POLKADOT	961
17	LUNCH BAG SUKI DESIGN	932
18	ALARM CLOCK BAKELIKE RED	917
19	WOODEN PICTURE FRAME WHITE FINISH	900
20	REX CASH+CARRY JUMBO SHOPPER	900
21	JUMBO BAG PINK POLKADOT	897
22	SET OF 4 PANTRY JELLY MOULDS	890
23	LUNCH BAG APPLE DESIGN	890
24	BAKING SET 9 PIECE RETROSPOT	885
25	RECIPE BOX PANTRY YELLOW DESIGN	883
26	JAM MAKING SET PRINTED	883
27	LUNCH BAG WOODLAND	850
28	ROSES REGENCY TEACUP AND SAUCER	844
29	VICTORIAN GLASS HANGING T-LIGHT	843

- 서비스 관련 정보를 포함하는 행들을 제거하기

```
service_removed_df = mysql("""
SELECT *
FROM filtered_df
WHERE UPPER(Description) NOT LIKE '%POSTAGE%'
      AND UPPER(Description) NOT LIKE '%CARRIAGE%'
      AND UPPER(Description) NOT LIKE '%BANK CHARGES%'
""")

print("before:", filtered_df.shape, "after:", service_removed_df.shape)
```

[결과 이미지를 넣어주세요]

```
before: (399689, 8) after: (399543, 8)
```

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기

```
standardized_df = mysql("""
SELECT
    InvoiceNo,
    StockCode,
    UPPER(Description) AS Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
```

```
Country
FROM service_removed_df
""")

print("before:", service_removed_df.shape, "after:", standardized_df.shape)
```

[결과 이미지를 넣어주세요]

```
before: (399543, 8) after: (399543, 8)
```

UnitPrice 살펴보기

- UnitPrice 의 최솟값, 최댓값, 평균을 구하기

```
mysql("""
SELECT
    MIN(UnitPrice) AS min_price,
    MAX(UnitPrice) AS max_price,
    AVG(UnitPrice) AS avg_price
FROM standardized_df
""")
```

[결과 이미지를 넣어주세요]

min_price	max_price	avg_price
0.0	649.5	2.904347

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최솟값, 최댓값, 평균 구하기

```
mysql("""
SELECT
    COUNT(*) AS cnt_quantity,
    MIN(Quantity) AS min_quantity,
    MAX(Quantity) AS max_quantity,
    AVG(Quantity) AS avg_quantity
FROM standardized_df
WHERE UnitPrice = 0
""")
```

[결과 이미지를 넣어주세요]

cnt_quantity	min_quantity	max_quantity	avg_quantity
33	1	12540	420.515152

- UnitPrice = 0 를 제거하고 일관된 데이터셋을 유지하기

```
price_clean_df = mysql("""
SELECT *
FROM standardized_df
WHERE UnitPrice <> 0
""")
```

```
print("before:", standardized_df.shape, "after:", price_clean_df.shape)
```

[결과 이미지를 넣어주세요]

```
before: (399543, 8) after: (399510, 8)
```

11-7. RFM 스코어

Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
invoice_day_df = mysql("""
SELECT
    DATE(
        printf(
            '%04d-%02d-%02d',
            CAST(
                substr(
                    substr(date_part, instr(date_part, '/') + 1),
                    instr(substr(date_part, instr(date_part, '/') + 1), '/') + 1
                ) AS INT
            ),
            CAST(substr(date_part, 1, instr(date_part, '/') - 1) AS INT),
            CAST(
                substr(
                    substr(date_part, instr(date_part, '/') + 1),
                    1,
                    instr(substr(date_part, instr(date_part, '/') + 1), '/') - 1
                ) AS INT
            )
        )
    ) AS InvoiceDay,
    *
FROM (
    SELECT
        *,
        substr(InvoiceDate, 1, instr(InvoiceDate, ' ') - 1) AS date_part
    FROM price_clean_df
) t
""")

invoice_day_df[["InvoiceDay", "InvoiceDate"]].head()
```

[결과 이미지를 넣어주세요]

	InvoiceDay	InvoiceDate
0	2010-12-01	12/1/2010 8:26
1	2010-12-01	12/1/2010 8:26
2	2010-12-01	12/1/2010 8:26
3	2010-12-01	12/1/2010 8:26
4	2010-12-01	12/1/2010 8:26

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
mysql("""
SELECT
    MAX(InvoiceDay) OVER () AS most_recent_date,
    InvoiceDay,
    *
FROM invoice_day_df
""")
```

[결과 이미지를 넣어주세요]

	most_recent_date	InvoiceDay	InvoiceDay	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	date_part
0	2011-12-09	2010-12-01	2010-12-01	536365	85123A	WHITE HANGING HEART T LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom	12/1/2010
1	2011-12-09	2010-12-01	2010-12-01	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom	12/1/2010
2	2011-12-09	2010-12-01	2010-12-01	536365	844068	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom	12/1/2010
3	2011-12-09	2010-12-01	2010-12-01	536365	840296	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom	12/1/2010
4	2011-12-09	2010-12-01	2010-12-01	536365	840296	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom	12/1/2010
...	...	...	...	...	...	...	...	...	...	...	...	...
399505	2011-12-09	2011-12-09	2011-12-09	581587	22613	PACK OF 20 SPACEBOY NAPKINS	12	12/9/2011 12:50	0.85	12680.0	France	12/9/2011
399506	2011-12-09	2011-12-09	2011-12-09	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	12/9/2011 12:50	2.10	12680.0	France	12/9/2011
399507	2011-12-09	2011-12-09	2011-12-09	581587	23254	CHILDREN'S CUTLERY DOLLY GIRL	4	12/9/2011 12:50	4.15	12680.0	France	12/9/2011
399508	2011-12-09	2011-12-09	2011-12-09	581587	23255	CHILDREN'S CUTLERY CIRCUS PARADE	4	12/9/2011 12:50	4.15	12680.0	France	12/9/2011
399509	2011-12-09	2011-12-09	2011-12-09	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	12/9/2011 12:50	4.95	12680.0	France	12/9/2011

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
customer_recent_df = mysql("""
SELECT
    CustomerID,
    MAX(InvoiceDay) AS InvoiceDay
FROM invoice_day_df
GROUP BY CustomerID
""")

customer_recent_df.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	InvoiceDay
0	12346.0	2011-01-18
1	12347.0	2011-12-07
2	12348.0	2011-09-25
3	12349.0	2011-11-21
4	12350.0	2011-02-02

- 가장 최근 일자(**most_recent_date**)와 유저별 마지막 구매일(**InvoiceDay**)간의 차이를 계산하기

```
recency_df = mysql("""
SELECT
    CustomerID,
    CAST(julianday(MAX(InvoiceDay) OVER ()) - julianday(InvoiceDay) AS INT) AS recency
FROM customer_recent_df
""")

recency_df.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	recency
0	12346.0	325
1	12347.0	2
2	12348.0	75
3	12349.0	18
4	12350.0	310

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```
user_r = mysql("""
SELECT
    a.CustomerID,
    a.InvoiceDay,
    b.recency
FROM customer_recent_df a
LEFT JOIN recency_df b
    ON a.CustomerID = b.CustomerID
""")

user_r.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	InvoiceDay	recency
0	12346.0	2011-01-18	325
1	12347.0	2011-12-07	2
2	12348.0	2011-09-25	75
3	12349.0	2011-11-21	18
4	12350.0	2011-02-02	310

Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```
user_f = mysql("""
SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM price_clean_df
GROUP BY CustomerID
""")

user_f.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt
0	12346.0	2
1	12347.0	7
2	12348.0	4
3	12349.0	1
4	12350.0	1

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```
user_item = mysql("""
SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
FROM price_clean_df
GROUP BY CustomerID
""")

user_item.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	item_cnt
0	12346.0	0
1	12347.0	2458
2	12348.0	2332
3	12349.0	630
4	12350.0	196

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
user_rf = mysql("""
WITH purchase_cnt AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT InvoiceNo) AS purchase_cnt
    FROM price_clean_df
    GROUP BY CustomerID
),
item_cnt AS (
    SELECT
        CustomerID,
        SUM(Quantity) AS item_cnt
    FROM price_clean_df
    GROUP BY CustomerID
)
SELECT
    pc.CustomerID,
    pc.purchase_cnt,
    ic.item_cnt,
    ur.recency
FROM purchase_cnt AS pc
```

```
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN user_r AS ur
  ON pc.CustomerID = ur.CustomerID
""")

user_rf.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency
0	12346.0	2	0	325
1	12347.0	7	2458	2
2	12348.0	4	2332	75
3	12349.0	1	630	18
4	12350.0	1	196	310

Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```
user_m = mysql("""
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 0) AS user_total
FROM price_clean_df
GROUP BY CustomerID
""")

user_m.head()
```

[결과 이미지를 넣어주세요]

	CustomerID	user_total
0	12346.0	0.0
1	12347.0	4310.0
2	12348.0	1437.0
3	12349.0	1458.0
4	12350.0	294.0

- 고객별 평균 거래 금액 계산

- 고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인(LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기

```
user_rfm = mysql("""
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
```



```

ut.user_total,
ROUND(1.0 * ut.user_total / rf.purchase_cnt, 2) AS user_average
FROM user_rf rf
LEFT JOIN user_m ut
ON rf.CustomerID = ut.CustomerID
""")

user_rfm.head()

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
0	12346.0	2	0	325	0.0	0.00
1	12347.0	7	2458	2	4310.0	615.71
2	12348.0	4	2332	75	1437.0	359.25
3	12349.0	1	630	18	1458.0	1458.00
4	12350.0	1	196	310	294.0	294.00

RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

```

mysql("""
SELECT *
FROM user_rfm
""")

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
0	12346.0	2	0	325	0.0	0.00
1	12347.0	7	2458	2	4310.0	615.71
2	12348.0	4	2332	75	1437.0	359.25
3	12349.0	1	630	18	1458.0	1458.00
4	12350.0	1	196	310	294.0	294.00
...	...	...	...	...	...	...
4357	18280.0	1	45	277	181.0	181.00
4358	18281.0	1	54	180	81.0	81.00
4359	18282.0	3	98	7	177.0	59.00
4360	18283.0	16	1355	3	2040.0	127.50
4361	18287.0	3	1586	42	1837.0	612.33

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) `user_rfm` 테이블과 결과를 합치기
- 3) `user_data` 라는 이름의 테이블에 저장하기

```

user_data = mysql("""
WITH unique_products AS (
    SELECT
        CustomerID,
        COUNT(DISTINCT StockCode) AS unique_products
    FROM price_clean_df
    GROUP BY CustomerID
)
SELECT
    ur.CustomerID,
    ur.purchase_cnt,
    ur.item_cnt,
    ur.rececy,
    ur.user_total,
    ur.user_average,
    up.unique_products
FROM user_rfm AS ur
JOIN unique_products AS up
    ON ur.CustomerID = up.CustomerID
""")

user_data.head()

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	rececy	user_total	user_average	unique_products
0	12346.0	2	0	325	0.0	0.00	1
1	12347.0	7	2458	2	4310.0	615.71	103
2	12348.0	4	2332	75	1437.0	359.25	21
3	12349.0	1	630	18	1458.0	1458.00	72
4	12350.0	1	196	310	294.0	294.00	16

2. 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)
 - 군 구매 소요 일수를 계산하고, 그 결과를 `user_data`에 통합

```

user_data = mysql("""
WITH invoice_days AS (
    SELECT DISTINCT
        CustomerID,
        InvoiceNo,
        InvoiceDay
    FROM invoice_day_df
    WHERE CustomerID IS NOT NULL
),
purchase_intervals AS (
    SELECT
        CustomerID,
        CAST(
            julianday(InvoiceDay) - julianday(
                LAG(InvoiceDay) OVER (
                    PARTITION BY CustomerID
                    ORDER BY InvoiceDay, InvoiceNo
                )
            ) AS INT
        ) AS interval_days
    FROM invoice_days
),

```

```

avg_intervals AS (
  SELECT
    CustomerID,
    CASE
      WHEN ROUND(AVG(interval_days), 2) IS NULL THEN 0
      ELSE ROUND(AVG(interval_days), 2)
    END AS avg_purchase_interval
  FROM purchase_intervals
  GROUP BY CustomerID
)
SELECT
  ud.CustomerID,
  ud.purchase_cnt,
  ud.item_cnt,
  ud.recency,
  ud.user_total,
  ud.user_average,
  ud.unique_products,
  ai.avg_purchase_interval
FROM user_data AS ud
LEFT JOIN avg_intervals AS ai
  ON ud.CustomerID = ai.CustomerID
""")

user_data.head()

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_purchase_interval
0	12346.0	2	0	325	0.0	0.00	1	0.00
1	12347.0	7	2458	2	4310.0	615.71	103	60.83
2	12348.0	4	2332	75	1437.0	359.25	21	94.33
3	12349.0	1	630	18	1458.0	1458.00	72	0.00
4	12350.0	1	196	310	294.0	294.00	16	0.00

3. 구매 취소 경향성

- 고객의 취소 패턴 파악하기
 - 1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
 - 2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
 - 취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data`에 통합하기
(취소 비율은 소수점 두번째 자리)

```

user_data = mysql("""
WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(DISTINCT CASE WHEN InvoiceNo LIKE 'C%' THEN InvoiceNo END) AS cancel_frequency
  FROM price_clean_df
  WHERE CustomerID IS NOT NULL
  GROUP BY CustomerID
)
SELECT
  u.CustomerID,
  u.purchase_cnt,
  u.item_cnt,
  u.recency,
  u.user_total,
  u.user_average,
  u.unique_products,

```

```

    u.avg_purchase_interval,
    t.total_transactions,
    t.cancel_frequency,
    ROUND(1.0 * t.cancel_frequency / t.total_transactions, 2) AS cancel_rate
FROM user_data AS u
LEFT JOIN TransactionInfo AS t
    ON u.CustomerID = t.CustomerID
""")

user_data.head()

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_purchase_interval	total_transactions	cancel_frequency	cancel_rate
0	12346.0	2	0	325	0.0	0.00	1	0.00	2	1	0.5
1	12347.0	7	2458	2	4310.0	615.71	103	60.83	7	0	0.00
2	12348.0	4	2332	75	1437.0	359.25	21	94.33	4	0	0.00
3	12349.0	1	630	18	1458.0	1458.00	72	0.00	1	0	0.00
4	12350.0	1	196	310	294.0	294.00	16	0.00	1	0	0.00

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기

```

mysql("""
SELECT *
FROM user_data
""")

```

[결과 이미지를 넣어주세요]

	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	avg_purchase_interval	total_transactions	cancel_frequency	cancel_rate
0	12346.0	2	0	325	0.0	0.00	1	0.00	2	1	0.50
1	12347.0	7	2458	2	4310.0	615.71	103	60.83	7	0	0.00
2	12348.0	4	2332	75	1437.0	359.25	21	94.33	4	0	0.00
3	12349.0	1	630	18	1458.0	1458.00	72	0.00	1	0	0.00
4	12350.0	1	196	310	294.0	294.00	16	0.00	1	0	0.00
...	...	...	...	...	...	...	...	...	...	...	...
4357	18280.0	1	45	277	181.0	181.00	10	0.00	1	0	0.00
4358	18281.0	1	54	180	81.0	81.00	7	0.00	1	0	0.00
4359	18282.0	3	98	7	177.0	59.00	12	59.50	3	1	0.33
4360	18283.0	16	1355	3	2040.0	127.50	262	22.27	16	0	0.00
4361	18287.0	3	1586	42	1837.0	612.33	59	79.50	3	0	0.00

회고

[회고 내용을 작성해주세요]

Keep : 지금의 학습 분위기를 계속 이어가야 한다. 아직 새로운 것을 많이 배우고 있지만, 조금씩 습관 익숙한 것도 있다. 이제까지 사용한 연습용은 데이터 크기가 많이 작다고 생각한다. 실제 업무에서는 어떤 크기의 데이터를 사용할까? 실수 없이 할 수 있어야 할텐데 걱정이야. 오늘도 한타 한타 키보드를 두드리자.

Problem : 대부분 처음 공부하는 것이고 비슷한 것도 많아서 헷갈린다. 그리고 금방 잊어버린다. 나중에 복습하면 비슷비슷하여 더욱 헷갈린다. 컴퓨터 환경도 아직 낯설다. 겨우 환경에 익숙하여 잘 사용할 만하면 다른 환경으로 변경한다. 낯선 환경에 낯선 코딩. 맨땅에 헤딩을 하더라도 문제만 없으면 좋겠다. 연습 기간에 충분히 실수하자. 처음의 대책 없는 헛발질에 당황한 것을 생각하면 지금도 웃음이 난다. 빨리 올챙이 시절을 지나 개구리가 되고 싶다.

Try : 눈으로만 보지 말고 손으로 계속 입력하면서 몸으로 익히자. 지금의 수 많은 오타가 실제에서는 시간 절약의 속성 과장이 되기를 희망한다. 지금까지 수 많은 오타를 통하여 통찰의 허탈함을 느꼈다. 모두 다 피가 되고 살이 되게, 국화 꽃을 피우는 과정이라고 생각하며 오늘도 키보드를 두드리자. 처음 기본문은 자동으로 입력되는 것이 너무 신기하였다. 이것이 몸으로 익힌 덕분이라고 생각한다. 그 다음은 주요 작업별 코딩이 자동으로 될 수 있도록 해야겠다. 이번 프로젝트의 과제 중 일부는 문제를 읽으면서 방법이 떠올랐으나 코딩을 해보니 어려가 생겼다. 코딩 방법에 아이디어가 생긴 것만으로도 기분이 좋다. 많은 과제에서 이 같은 아이디어가 생겼으면 좋겠다. 오늘 익힌 것을 일주일후 또 한달후에 다시 반복 반복하여 연습해야겠다.